

set_operating_mode

REFERENCE

API consumer

configure the reader's RFID operating mode

Overview

`set_operating_mode` configures the reader's RFID operating mode — the profile, advanced RF configuration, tag access operations, radio start/stop triggers, the Gen2 `query`, select pre-filters, and tag-metadata reporting flags. It is a state-changing **control-plane (CTRL)** command. It is a **partial, merging** update: you send only the sub-objects you want to change and they merge into the current mode. Read the result back with `get_operating_mode`.

`accessOperations` arm destructive tag operations (`WRITE` / `LOCK` / `KILL`) that execute on the next inventory — `KILL` and `PERMANENT_LOCK` are irreversible. See the danger callout in [Notes and Caveats](#) before sending.

This is a **CTRL-endpoint command**, published on the **control** endpoint (`<tenantId>/CTRL/clients/cmnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel). This **requires an active CTRL endpoint** (`config_endpoint`, `epType: CTRL`, `activate: true`); without one, you get no response.

Set the [regulatory region](#) first. Before you start an RFID inventory, make sure the reader's regulatory region is configured: the reader does not transmit until it is, and the region is set outside MQTT with 123RFID Desktop or 123RFID Mobile. Use `get_current_region` to verify the current region.

Details

Property	Value
Interaction Pattern	Request / Response (partial-merge configuration)
State Change	STATE-CHANGING (arms access ops; changes RF config)
Reversibility	REVERSIBLE / VOLATILE (reset to default on <code>reboot</code> , or re-send to change)
Required Request Fields	<code>command</code> , <code>requestId</code> (<code>operatingMode.operatingModes</code> is optional in the contract; send it with the aspects to set)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_operating_mode</code> , <code>control_operation</code> , <code>config_endpoint</code> , <code>reboot</code>

When to Use

Use `set_operating_mode` to configure how the reader reads (and acts on) tags:

- **Select a profile** — `CYCLE_COUNT` (read as many unique tags as possible), `DENSE_READERS` (multiple readers in proximity), `OPTIMAL_BATTERY` (best battery life), `BALANCED_PERFORMANCE` (default; balances performance and battery life), or `ADVANCED` (custom link-profile settings via `advancedConfigurations`). `FAST_READ` (maximum read speed within short range) is enumerated but not currently supported.
- **Tune the radio** — transmit power, link profile, session, dynamic power (`ADVANCED`).
- **Compose triggers** — `radioStartConditions` / `radioStopConditions` (trigger, delay, tag/time/cycle stops).
- **Shape inventory** — Gen2 `query` and up to 32 `select` pre-filters.
- **Choose reporting** — `tagMetaDataToEnable` flags.
- **Arm tag operations** — `accessOperations` (`READ/WRITE/ACCESS/LOCK/KILL`) that run on the next inventory. Handle with care (see the danger note).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Control commands are answered **only on the CTRL endpoint**.

If control commands time out, confirm a CTRL endpoint is active (`get_endpoint_config` → `activeEndpoints`) and activate one with `config_endpoint` if needed. The mode cannot be

changed while an inventory is running ([code 11](#)).

See [Topics and Channels](#).

Key Fields

- `operatingModes.profiles` (request) — the RF profile; `ADVANCED` requires `advancedConfigurations`. `FAST_READ` is the first member of the enum but is not currently supported, and data events are not supported while the sled is in it.
- `operatingModes.accessOperations` (request) — ⚠ armed tag operations (destructive; see the danger note).
- `operatingModes.select` (request) — up to 32 Gen2 pre-filters.
- `operatingModes.query` / `radioStartConditions` / `radioStopConditions` (request) — inventory shaping and trigger semantics.
- `operatingModes.tagMetadataToEnable` (request) — which metadata fields the reader reports on each read. The switch-to-field mapping is on [dataEVT](#).
- `response.code` (response) — read this to determine success vs failure; the contract requires it; note the reported success-path omission (see [Response Payload](#)) and the partial-apply-on-error behavior (see [Request Semantics](#)). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `operatingMode` payload. Send only the `operatingModes` sub-objects you want to change. For a basic inventory, start with **Profile setting**: it sets only the RF profile. **Set operating mode** also sets advanced RF settings and arms tag access operations (`READ`, `WRITE`, `ACCESS`, `LOCK`), which run on the next inventory `START`; send it only when you want those operations. Pick a **configuration section** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Configuration section

Profile setting ▼

Request payload

Example: basic inventory: profile only

```
1  {
2    "command" : "set_operating_mode" ,
3    "requestId" : "1286" ,
4    "operatingMode" : {
5      "operatingModes" : {
6        "profiles" : "BALANCED_PERFORMANCE"
7      }
8    }
9  }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAM
1	command *	enum	set_operating_mode	set
2	requestId *	string	—	128
3	operatingMo de	object	—	—
4	operating Modes	object	—	—
5	profil es	enum	FAST_READ CYCLE_COUNT DENSE_READERS OPTIMAL_BATTERY BALANCED_PERFORMANCE ADVANCED	ADV
6	advanc edConf igurat ions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
7	transmitPower	number	0-300	270
8	linkProfile	enum	M4_256K M2_240K M2_256K M2_320K M4_240K M4_320K FM0_0K FM0_320K M8_240K M8_256K M8_320K	M2_
9	session	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SES
10	dynamicPower	boolean	—	true
11	inventoryProtocol	enum	GEN2 GEN2X HYBRID	GEN
12	accessOperations	array	—	—
13	operationType	enum	READ WRITE ACCESS LOCK KILL	REA
14	config	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
15	m e m o r y B a n k	enum	EPC TID USER RESERVED	EPC
16	o f f s e t	integer	—	1
17	l e n g t h	integer	—	2
18	d a t a	string	—	FFF
19	p a s s w o r d	string	—	000

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
20	lockMemBank	enum	EPC ACCESS_PWD KILL_PWD TID USER ALL	TID
21	lockActivation	enum	READ_AND_WRITE PERMANENT_LOCK PERMANENT_UNLOCK UNLOCK	UNLOCK
22	radioStartConditions	object	—	—
23	trigger	enum	PRESSED RELEASED IMMEDIATE	IMMEDIATE
24	startDelay	integer	—	300
25	repeat	boolean	—	false
26	radioStopConditions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
27	tri gge r	enum	PRESSED RELEASED IMMEDIATE	IMM
28	tag Cou nt	integer	—	10
29	sto pTi meo ut	integer	—	100
30	inv ent ory Cou nt	integer	—	0
31	query	object	—	—
32	ses sio n	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SES
33	inv ent ory Sta te	enum	STATE_A STATE_B STATE_AB	STA
34	slF lag	enum	ALL ASSERTED DEASSERTED	ALL
35	tag Pop ula tio n	integer	min 0	30
36	select	array	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
37	enable *	boolean	—	true
38	tagParameter *	string	—	222
39	memoryBank *	enum	EPC TID USER	EPC
40	offset *	integer	—	32
41	action *	enum	INV_A_NOT_INV_B_OR_ASRT_SL_NOT_DSRT_SL INV_A_OR_ASRT_SL NOT_INV_B_OR_NOT_DSRT_SL INV_A2BB2A_NOT_INV_A_OR_NEG_SL_NOT_ASRT_SL INV_B_NOT_INV_A_OR_DSRT_SL_NOT_ASRT_SL INV_B_OR_DSRT_SL NOT_INV_A_OR_NOT_ASRT_SL NOT_INV_A2BB2A_OR_NOT_NEG_SL	INV
42	tagParameter *	enum	SESSION_SL SESSION_0 SESSION_1 SESSION_2 SESSION_3	SES
43	length *	integer	—	16
44	tagMetadataObjectEnable	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAM
45	RSSI	boolean	—	false
46	PHASE	boolean	—	false
47	SEENCOUNT	boolean	—	false
48	CHANNEL	boolean	—	false
49	PC	boolean	—	false
50	XPC	boolean	—	true
51	CRC	boolean	—	true
52	EPC	boolean	—	true
53	TID	boolean	—	true
54	USER	boolean	—	true
55	MAC	boolean	—	true
56	HOSTNAME	boolean	—	true

Request Semantics

Merge. `set_operating_mode` is **cumulative**: it updates only the aspects present in the payload and leaves all others intact — it is **not** a full replacement. For example, sending only `radioStartConditions` changes the triggers while preserving `profiles`, `query`, `select`, `accessOperations`, and `tagMetadataToEnable`. Send `{profiles: "BALANCED_PERFORMANCE"}` alone to reset the profile without touching anything else. Because merges accumulate, **armed** `accessOperations` **persist** across later partial sets until you overwrite them or reboot. Confirm the resulting merged mode with `get_operating_mode`.

- The singular `operatingMode` wrapper around the plural `operatingModes` object is the schema-defined shape. The command's `operatingMode` property carries a wrapper holding exactly one member, `operatingModes`, and every worked example nests it that way — the double nesting is intentional and unique to this command (see the Glossary). The contract marks neither level `required` and does not forbid unrecognized properties, so a flattened or renamed payload still validates — but it does not carry the operating mode where the schema defines it. Send the nested shape exactly as shown.
- An error response can leave the mode partially applied. An error response (e.g. `101` "Error in processing command") can leave the mode **partially applied** — always verify the result with `get_operating_mode` rather than assuming a `101` changed nothing.
- **ADVANCED** requires `advancedConfigurations`; **limits are enforced**. `profiles: ADVANCED` without `advancedConfigurations` returns `code 22`. `select` supports up to 32 pre-filters (`code 24` if exceeded); an overlong `select.tagPattern` returns `28`; **WRITE** `data` that is too large or mismatched returns `31` / `32`. `FAST_READ` is enumerated but **not currently supported**, and configuring `FAST_READ` disables data events.

Response Payload

`set_operating_mode` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`.

Response payload

```
1 {
2   "command" : "set_operating_mode" ,
3   "requestId" : "abc123" ,
4   "apiVersion" : "V1.1" ,
5   "response" : {
6     "code" : 0 ,
7     "description" : "Success"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command * mand d	string	—	set_operating_mode	Echoes the executed command.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — set_operating_mode echoes no configuration.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Payload and Response Codes below.
6	description *	string	—	Success	Human-readable status.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful set can arrive with `code` omitted. This form does not validate against the contract; the reply with `code: 0` above is the documented form:

Response — success reported without code (not accepted by the contract)

```
1  {
2    "command" : "set_operating_mode" ,
3    "requestId" : "1286" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "description" : "Success"
7    }
8  }
```

Response Codes

`set_operating_mode` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the mode was applied. (Readers have been reported to omit <code>code</code> on this reply — see above.)
11	Inventory in progress (<code>IOT_ERROR_INVENTORY_IN_PROGRESS</code>) — the mode cannot be changed while an inventory is running. Stop it with control_operation first.
22	Advanced configuration not set (<code>IOT_CTRL_ADVANCED_PROFILE_NOT_SET</code>) — <code>profiles:ADVANCED</code> was sent without <code>advancedConfigurations</code> .
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (e.g. <code>profiles</code> , <code>operationType</code> , a Gen2 enum) is outside its allowed set.
24	Max 32 prefilters limit exceeded (<code>IOT_ERROR_PREFILTERS_LIMIT_EXCEEDED</code>) — more than 32 <code>select</code> entries.
28	Tag match pattern length exceeded (<code>IOT_ERROR_TAG_MATCH_PATTERN_LENGTH_EXCEEDED</code>) — a <code>select.tagPattern</code> is too long.
31	Max write length exceeded (<code>IOT_ERROR_MAX_WRITE_LENGTH_EXCEEDED</code>) — a WRITE <code>data</code> block is too large.
32	Write data length mismatch (<code>IOT_ERROR_WRITE_DATA_LENGTH_MISMATCH</code>) — a WRITE <code>data</code> length does not match its <code>length</code> .

Code	Description
101	Default error (<code>Error in processing command</code>) — a generic failure. Note: a 101 can still leave the mode <i>partially applied</i> — read back with <code>get_operating_mode</code> to see what took effect.

Examples

Publish `set_operating_mode` over the CTRL endpoint; the device replies with the status envelope under [Response Payload](#). Every request example in the contract is in the **Configuration section** dropdown under [Request Payload](#): the full `ADVANCED` configuration (**Set operating mode**, which arms READ, WRITE, ACCESS and LOCK operations with the default password `00000000`), and one partial set per section. **Profile setting** is also the way to return the profile to its default.

A generic failure returns [code](#) `101` (which may still partially apply — read back to confirm):

Response — error (code 101)

```

1      {
2          "command" : "set_operating_mode" ,
3          "requestId" : "1286" ,
4          "apiVersion" : "V1.1" ,
5          "response" : {
6              "code" : 101 ,
7              "description" : "Error in processing command"
8          }
9      }
```

Notes and Caveats

⚠️ ARMED ACCESS OPERATIONS EXECUTE ON THE NEXT INVENTORY — AND THE PAYLOAD CARRIES THEIR PASSWORD

`accessOperations` (`WRITE` , `LOCK` , `KILL`) are stored by `set_operating_mode` and executed against tags **on the next inventory START** — not immediately. `KILL` permanently disables a tag; `LOCK` with `PERMANENT_LOCK` / `PERMANENT_UNLOCK` is irreversible; `WRITE` overwrites tag memory. Because partial sets **merge**, armed operations persist until you **overwrite the** `accessOperations` **array** (with a benign or empty set) or `reboot` . A profile-only set — or any partial set that omits `accessOperations` — does **not** clear them. Do this before running an inventory that should not trigger them. Each armed operation also carries `accessOperations[].config.password` , an 8-hex-char (32-bit) access/lock/kill password you send (default `00000000`) — mask it in shared artifacts. The device stores it but never echoes it back (`get_operating_mode` returns it empty). This command carries no broker/Wi-Fi credentials.

- **Volatile and CTRL-only.** The operating mode resets to default on `reboot` , and the command is answered only on the CTRL endpoint (see [Topic Mapping](#)).
- `FAST_READ` **is enumerated but not currently supported.** `FAST_READ` is the first member of the `profiles` enum, and the schema glosses it as "Configures the radio for maximum read speed within short range. (Currently not supported)". The command description repeats the restriction and extends it to the event plane: "When RFD40/90 sled is configured to FAST_READ profile, data events are not currently supported. On reboot the set configurations will be lost and the device will go back to default operating mode." Treat `FAST_READ` as unavailable, and expect no `dataEVT` traffic while the sled is in that profile.

Configure with `set_operating_mode` , then **verify with** `get_operating_mode` (especially after a `101`), and run the inventory with `control_operation` . For the concept-level view, see [Operating Mode](#).

Related:  `get_operating_mode` ·  `control_operation` ·  `config_endpoint` ·  `reboot` ·  [Response Codes](#)